Advanced Retrieval I: Hybrid Search

Modulo 3 · Lezione 4



Il punto di partenza: recupera() cerca per significato

`indicizza()`

`recupera(query, k)`

embedding → coseno → top-k

`genera(chunks, domanda, storia)`

`rispondi(query, storia)`

scaffolding/aikit/rag.py — da due lezioni nel toolkit

~~chat.py~~ stasera chiuso: single-turn fino a Modulo 3 · Lezione 9

DAL CATALOGO DI MODULO 3 · LEZIONE 1

«La scrivania non sta ferma, si muove tutta quando scrivo» → **retrieval miss**: il chunk giusto era al posto 12. La risposta: «Non lo trovo nei documenti».

> Stasera la prima cura del retrieval: una **seconda ricerca** accanto a quella per significato, e la **fusione** delle due.



Agenda

	Blocco	Dove	Tempo
	Dove il semantico sbaglia, TF-IDF, BM25, la fusione, RRF	slide	45′
	Codice spiegato: l'indice BM25 (già nel file)	`working/hybrid.py`	12′
	Esercizio: `rrf()` e `cerca_hybrid()` (TODO 3-4)	`working/hybrid.py`	45′
	Reveal e wrap-up	slide	18′

Un solo file per tutta la serata: `working/hybrid.py`. Si lancia dalla cartella `scripts/`: `python working/hybrid.py`.



Dove il semantico sbaglia: due query dal corpus

```
D: Cosa è successo nel ticket TCK-1152?

 1. ticket-supporto-012  0.550  TCK-1228
 2. ticket-supporto-002  0.545  TCK-1103
 3. ticket-supporto-001  0.537  TCK-1042
 …                              altri sei ticket
10. ticket-supporto-008  0.470  ← TCK-1152, il giusto
```

```
D: Cosa copre SRV-02?

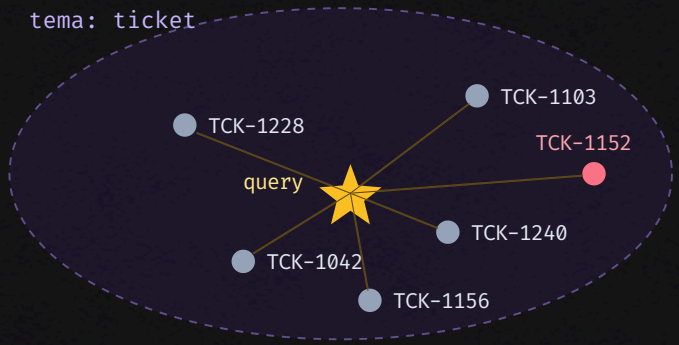
1. policy-resi-garanzia-009 garanzia SRV-03
2. ticket-supporto-012      cita SRV-01
3. listino-catalogo-006     ← SRV-02, il giusto
```

Il chunk giusto **c'è**: decimo, terzo. Nessuna delle righe davanti è sbagliata per caso: tutte parlano di ticket, di servizi.

Con k=3 «decimo» vuol dire **fuori**. La risposta: «Non lo trovo nei documenti».



Perché l’embedding non pesa il codice



- tutti i ticket hanno lo **stesso tema**
- coseni **piatti**: 0.550 · 0.545 · 0.537
- il codice **non pesa** nel vettore

TCK-1152 non ha sinonimi né vicini semantici. L'embedding del chunk porta con sé il tema del paragrafo, non il codice.

> **Misranking, non miss**: il chunk c'è, decimo, dietro nove ticket. Ma con k piccolo, l'ordine è tutto.

Cosa costruiamo stasera: i pezzi e il prodotto

`cerca_hybrid(query, k)` il prodotto finale: **un solo ranking** da due ricerche — TODO 4

RANKING PER SIGNIFICATO · già in aikit/

`rag.recupera(query)`

embedding della domanda → coseno con i chunk → top-k. Non si tocca.

RANKING PER PAROLE · già nel file, codice spiegato

`cerca_bm25(query)`

token della domanda → score **BM25** per ogni chunk → top-k.

BM25 = TF-IDF + saturazione + lunghezza del chunk

TF-IDF non è un pezzo a parte: è la versione base di BM25, si vede per capire BM25.

`rrf(liste)` La **fusione**: prende i due ranking e ne fa uno usando solo le posizioni, mai gli score. È il concetto della serata. — TODO 3

Ordine delle slide: prima BM25 (da TF-IDF), poi perché le due liste non si sommano, poi RRF.



Keyword search: token esatti

```
# già nel file: minuscole, solo lettere e numeri
def tokenizza(testo):
    return re.findall(r"\w+", testo.lower())
```

«Cosa copre SRV-02?» → `cosa` `copre` `srv` `02`

BM25 confronta **questi token** con quelli dei chunk. Esatti: niente sinonimi, niente parafrasi.

ATTENZIONE

`2.3.0` → `2` · `3` · `0`: tre token presenti in 17, 8 e 9 chunk. Il tokenizer decide cos'è una parola.



TF-IDF: la formula

$$\text{score}(\text{query}, \text{chunk}) = \sum_{t \in \text{query}} \text{tf}(t, \text{chunk}) \cdot \text{idf}(t)$$

`tf(t, chunk)` · TERM FREQUENCY

Quante volte il token `t` compare nel chunk. Tre volte → 3.

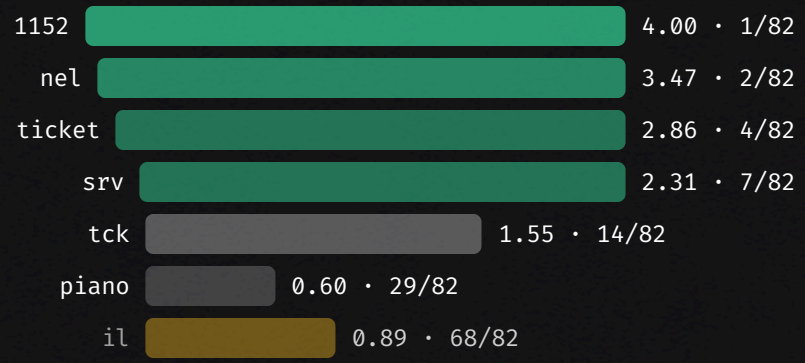
`idf(t)` · INVERSE DOCUMENT FREQUENCY

$\ln\left((N - n_t + 0.5) / (n_t + 0.5) \right)$

$N = 82$ chunk · n_t = in quanti chunk sta `t`. Raro → pesa tanto; ovunque → non distingue.

Si somma solo sui token della domanda: un chunk che non ne contiene nessuno fa 0.

IDF NEL NOSTRO INDICE (N = 82)



«il» sta in 68 chunk su 82: la formula dà `-1.55`. La libreria (`rank-bm25`) i valori negativi li rialza a un minimo, 0.89. «nel» invece in questo corpus è raro, e pesa: si vede nella prossima slide.

TF-IDF sulla demo: il chunk giusto è terzo

«Cosa è successo nel ticket TCK-1152?» → token con un peso: è 0.60 · nel 3.47 · ticket 2.86 · tck 1.55 · 1152 4.00 (cosa e successo non stanno in nessuno dei tre chunk).

1. ticket-supporto-000

l'intestazione della knowledge base

ticket	3 × 2.86 = 8.58

TF-IDF = 8.58

2. guida-smart-working-008

non parla di ticket

nel	2 × 3.47 = 6.94
è	1 × 0.60 = 0.60

TF-IDF = 7.54

3. ticket-supporto-008 ← il giusto

TCK-1152, il chunk che cerchiamo

1152	1 × 4.00 = 4.00
tck	1 × 1.55 = 1.55
è	1 × 0.60 = 0.60

TF-IDF = 6.14

> Il problema è **tf**: tre «ticket» valgono tre volte uno, due «nel» valgono due volte uno. Una parola ripetuta batte il codice esatto. Serve un correttivo: **BM25**.



BM25: TF-IDF con saturazione e lunghezza

$$\text{score} = \Sigma_t \ \text{idf}(t) \cdot \text{tf} \cdot (k_1 + 1) \ / \ (\ \text{tf} + k_1 \cdot (1 - b + b \cdot |\text{chunk}| \ / \ \text{media})\ ) \qquad k_1 = 1.5 \cdot b = 0.75$$

SATURAZIONE · tf non conta più linearmente

tf = 1 → 1.00 · 2 → 1.43 · 3 → 1.67 · 10 → 2.17

La terza occorrenza vale un quarto della prima. Ripetere una parola non fa vincere.

LUNGHEZZA · a parità di tf, il chunk corto pesa di più

metà della media → 1.29 · media → 1.00 · doppia → 0.69

Media del nostro indice: 57 token per chunk.

1. ticket-supporto-008 ← il giusto

1152	4.00 → 3.72
tck	1.55 → 1.45
è	0.60 → 0.55

BM25 = 5.72

2. guida-smart-working-008

nel ×2	6.94 → 4.38
è	0.60 → 0.50

80 token: sopra la media

BM25 = 4.88

3. ticket-supporto-000

ticket ×3	8.58 → 4.39

76 token: sopra la media

BM25 = 4.39

Stessi tre chunk, stesse parole: le ripetizioni saturano e il codice esatto vince. Lo score resta una somma senza massimo.



Come si compone lo score TF-IDF

un **token** della domanda + il **chunk** → **tf**: quante volte il token sta nel chunk — `1152 in ticket-supporto-008: 1 volta`

un **token** della domanda + il **corpus**, 82 chunk → **idf**: in quanti chunk del corpus sta il token — `1152: in 1 chunk su 82 → pesa 4.00`

tf × **idf** → il **peso** di quel token in quel chunk — `1152: 1 × 4.00 = 4.00`

i pesi dei token che stanno **sia nella domanda sia nel chunk** Σ → lo **score TF-IDF** del chunk — `1152 + tck + è → 6.14`

lo score di **ogni** chunk ordina → il **ranking** per parole — `il giusto: terzo`

Un token pesa per quanto è **frequente nel chunk** e **raro nel corpus**. I token della domanda che nel chunk non ci sono contribuiscono zero.



Come si compone lo score BM25

token + chunk → **tf** → **tf saturato**: ogni ripetizione in più conta meno — `ticket ×3: non vale più 3 volte`

chunk + corpus → **lunghezza** del chunk rispetto alla media — `chunk lungo → pesa meno`

token + corpus → **idf**: in quanti chunk sta il token — `come in TF-IDF`

idf × **tf saturato** · correzione per **lunghezza** → il **contributo** del token — `1152: 4.00 → 3.72`

i contributi dei token che stanno **sia nella domanda sia nel chunk** Σ → lo **score BM25** del chunk — `→ 5.72`

lo score di **ogni** chunk ordina → il ranking di `cerca_bm25()` — `il giusto: primo`

Stessa struttura di TF-IDF: cambia solo come si arriva al peso di un token, con la saturazione e la lunghezza in mezzo.



BM25 sulla demo, e cosa non perdona

```
D: Cosa è successo nel ticket TCK-1152?

 1. ticket-supporto-008  5.72  TCK-1152
 2. guida-smart-working-008 4.88  «nel» due volte, raro nel corpus
 3. ticket-supporto-000  4.39  «ticket» tre volte, saturato

lo score non ha un massimo: è una somma
```

NON PERDONA · STESSE PAROLE

«Perché il piano si ferma sempre a metà corsa?» → tutti i ticket hanno queste parole: il risolutivo è **quarto**.

NON PERDONA · SINONIMI

«Ho comprato con un codice sconto: posso restituire?» → il chunk dice «saldi», «codice promozionale»: **decimo**.

> Le due ricerche sbagliano **in posti diversi**: è la ragione per tenerle entrambe.

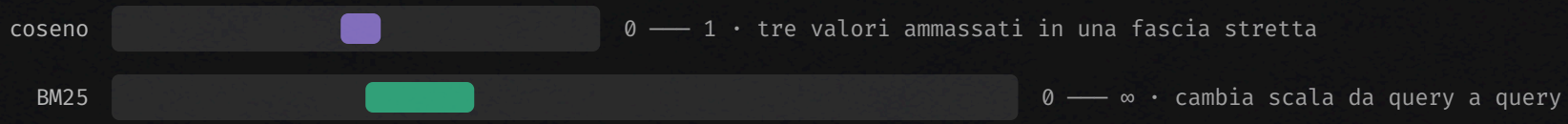


Due liste, due scale: gli score non si sommano

```
[semantico] coseno
1. ticket-supporto-012  0.550
2. ticket-supporto-002  0.545
3. ticket-supporto-001  0.537
10. ticket-supporto-008 0.470
```

+

```
[bm25] score
1. ticket-supporto-008  5.72
2. guida-smart-working-008 4.88
3. ticket-supporto-000  4.39
…
```



Tre termini rari fanno 14, termini comuni fanno 4: BM25 non ha un massimo. Il coseno vive tra 0 e 1. Sommarli non ha senso.

Perché normalizzare è fragile

```
score_finale = 0.5 · coseno_normalizzato + 0.5 · bm25_normalizzato
```

LA SCALA DIPENDE DALLA LISTA

Il min-max si calcola sui risultati di **questa** query: il massimo di oggi non è quello di domani.

UN OUTLIER SCHIACCIA GLI ALTRI

Un BM25 a 22 su una lista che sta tra 4 e 6 manda tutti gli altri vicino a zero.

I PESI VANNO TARATI

0.5 e 0.5? 0.7 e 0.3? Per deciderlo servono le metriche — Modulo 3 · Lezione 10.

Si può fare, e qualcuno lo fa. Ma c'è un modo che non ha nessuno di questi tre problemi.



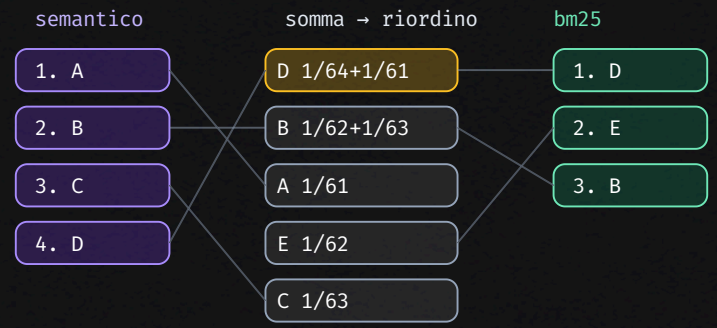
RRF: solo il rango

$$\text{score}(\text{chunk}) = \Sigma_{\text{liste}}\ 1 / (60 + \text{posizione})$$

Si buttano via gli score, si tiene la **posizione**. Ogni chunk vale `1/(60+pos)` in ogni lista in cui compare; i contributi si **sommano**; si riordina per somma.

`1/61 = 0.0164 · 1/62 = 0.0161 · 1/70 = 0.0143`

Il 60 è una costante empirica (`CONFIG["rrf_costante"]`): appiattisce le differenze tra le posizioni alte, così un primo posto isolato non domina.



RRF sulla demo, a mano

```
ticket-supporto-008 (TCK-1152)
 semantico: 10° → 1/(60+10) = 0.0143
 bm25:      1°  → 1/(60+1)  = 0.0164
 somma:                  0.0307  → primo

ticket-supporto-012 (TCK-1228)
 semantico: 1°  → 1/61 = 0.0164
 bm25:      —   → 0
 somma:                 0.0164
```

RRF premia chi compare in **entrambe** le liste.

`5° e 6°: 1/65 + 1/66 = **0.0305**`
`1° in una lista sola: 1/61 = **0.0164**`

La concordanza vale più di un primo posto isolato. Quasi sempre è giusto; ha un costo, e lo vedremo nella tabella.



Quando l’hybrid vince e quando è peso morto

VINCE

codici prodotto, sigle di servizio, numeri di ticket

nomi propri, versioni, gergo interno

tutto ciò che è un **token esatto senza sinonimi**

PESO MORTO

domande concettuali («posso lavorare dall'estero?»)

parafrasi, sintomi descritti con parole proprie

dove il semantico **basta già**

L'hybrid **non deve far uscire dalla top-3** ciò che funzionava. Se una domanda concettuale scende di un posto restando dentro, il costo c'è comunque — e va detto.



Il costo: un indice in più

collection lumen_m3
embedding · ChromaDB

↔
stessi 82 chunk
stessi id

INDICE BM25
token · rank-bm25, in memoria

DUE INDICI DA TENERE ALLINEATI

Si re-indicizza la collection → si ricostruisce BM25. Cambiano i chunk → entrambi. Stasera è una funzione che gira in una frazione di secondo; in produzione è un secondo sistema.

I vector DB gestiti offrono l'hybrid nativo: stessa idea, chiusa in un servizio. Qui lo scriviamo noi, per vedere cosa succede dentro.



Il query set fisso del modulo

6 CON UN CODICE

«Cosa copre SRV-02?»

«Com'è stato risolto il TCK-1201?»

«Cosa ha introdotto il firmware 2.3.0?»

6 CONCETTUALI

«Posso lavorare dall'estero?»

«Il monitor scende piano piano da solo»

«La scrivania non sta ferma…» (dal catalogo)

1 FUORI CORPUS

«Posso pagare alla consegna, in contrassegno?»

Modulo 3 · Lezione 4 — stesse tredici domande — Modulo 3 · Lezione 6 · Modulo 3 · Lezione 10: golden set

Il giudizio è ancora **a occhio**, ma è **ripetibile**: stessa domanda, stesso esito, prima e dopo.

Le domande sono già nel file (QUERY_SET). Il chunk giusto lo trovate voi, aprendo i chunk stampati.



Codice spiegato: l'indice BM25

CODICE SPIEGATO — il docente spiega, voi seguite sul vostro file

`working/hybrid.py` · funzioni 1-2, già scritte · 12 minuti

si rientra dopo il lancio, con la demo a schermo

1 · `costruisci_indice_bm25()`

tutti i chunk dalla collection (`collection.get()`) → `tokenizza()` → `BM25Okapi` → INDICE

2 · `cerca_bm25(query, k)`

`get_scores(tokenizza(query))` → ordina → i primi k, nella forma di `rag.recupera()`

Le due funzioni sono già nel vostro file: si segue, non si ricopia. C'è già anche `CONFIG`, `QUERY_SET`, `tokenizza()` e il main. L'indice si costruisce in una frazione di secondo, senza chiamate.



Cosa si è visto: la demo fianco a fianco

```
$ python working/hybrid.py
indice BM25: 82 chunk, 1394 termini distinti

D: Cosa è successo nel ticket TCK-1152?

  [semantico]
    1. ticket-supporto-012  score=  0.550  TCK-1228 · SRV-01 · appunt…
    2. ticket-supporto-002  score=  0.545  TCK-1103 · LD-210 · il piano…
    3. ticket-supporto-001  score=  0.537  TCK-1042 · LD-200 · il piano…

  [bm25]
    1. ticket-supporto-008  score=  5.720  TCK-1152 · LD-115 · morsetto…
    2. guida-smart-working-008 score=  4.880  Nelle giornate da remoto…
    3. ticket-supporto-000  score=  4.390  KNOWLEDGE BASE — TICKET…
```

DUE SCALE

Coseni sopra 0.5 da una parte, 5.72 dall'altra. Il chunk giusto: decimo a sinistra, primo a destra.

ORA LA FUSIONE

Non si sommano. È l'esercizio: TODO 3-4.



Esercizio: rrf() e cerca_hybrid()

SCRIVETE VOI — da soli, con affiancamento

`working/hybrid.py` · `TODO 3-4` · `45 minuti` · `consegna in CONSEGNA.md`

si rientra per il reveal

`TODO 3 · rrf(liste, k)`

per ogni lista, per ogni chunk: `1/(costante + posizione)`, sommato per id → ordina → i primi k

`TODO 4 · cerca_hybrid(query, k)`

dieci candidati dal semantico, dieci da BM25 → `rrf()` → k

`FATTO QUANDO`

Il main stampa le tre modalità per tutto il query set, e nella tabella in fondo al file: sulle domande con codice l'hybrid non è mai dietro al semantico, e sulle concettuali il chunk giusto resta in top-3. Lo stesso id due volte nell'hybrid: `rrf()` ricostruisce i chunk male. Due errori non si vedono a occhio (score al posto della posizione, k al posto di `candidati`): per quelli, `python scaffolding/check_lab04.py`. Chi finisce: candidati a 3 e a 30, costante a 1 — cosa cambia, e su quali domande.



Reveal: la tabella

#	query	chunk giusto	sem	bm25	hybrid
1	…ticket TCK-1152?	ticket-supporto-008	10	1	1
2	Cosa copre SRV-02?	listino-catalogo-006	3	2	2
3	…risolto il TCK-1201?	ticket-supporto-004	3	1	1
4	Con l'errore E07…	manuale-lumadesk-009	1	1	1
5	Quale RMA… danno da trasporto?	listino-catalogo-008	1	1	1
6	…firmware 2.3.0?	changelog-rilasci-002	1	2	1
7	La scrivania non sta ferma…	manuale-lumadesk-013	—	—	—
8	Quanto tempo ho… contributo…	guida-smart-working-004	1	9	3
9	Posso lavorare dall'estero?	guida-smart-working-009	1	1	1
10	Il monitor scende piano piano…	ticket-supporto-007	1	1	1
11	…codice sconto: posso restituire?	policy-resi-garanzia-007	1	10	2
12	…si ferma sempre a metà corsa?	ticket-supporto-003	1	4	2
13	…in contrassegno? (fuori corpus)	—	—	—	—



Cosa dice la tabella

STACCA

TCK-1152 10 → 1. TCK-1201 3 → 1: terzo nel semantico, primo in BM25, la somma lo porta in testa. È la concordanza.

NON CAMBIA

E07, RMA, 2.3.0, estero, monitor: primo in tutte e tre (sul 2.3.0 BM25 da solo era secondo: il tokenizer). Il semantico bastava; l'hybrid non ha rotto niente.

IN TOP-3, NON IN TESTA

SRV-02: hybrid secondo. Davanti, in tutte e tre, la garanzia SRV-03: prima in entrambe le liste, `1/61 + 1/61 = 0.0328` contro `1/63 + 1/62 = 0.0320`. Se le due ricerche concordano sul chunk sbagliato, la fusione non cura.

LE CONCETTUALI SCENDONO

Contributo 1 → 3, codice sconto 1 → 2, metà corsa 1 → 2: un chunk vicino, in entrambe le liste, scavalca il giusto che in BM25 stava in fondo. Nessuna esce dalla top-3 — ma è il costo, e si dice.

Non si cura stasera: «La scrivania non sta ferma» è fuori in tutte e tre — una parafrasi (→ Modulo 3 · Lezione 13). Il contrassegno: nessun chunk giusto, il rifiuto lo dà `rispondi()`.



La prossima lezione

`SRV-02: hybrid 2°` `contributo: hybrid 3°` → **Modulo 3 · Lezione 5 — Advanced Retrieval II: Re-ranking**

L'hybrid porta il chunk giusto nella top-3 su tutte le query con codice, ma su SRV-02 non al primo posto, e su tre concettuali lo fa scendere. La prossima lezione prende quella top-10 e la **riordina** con un modello che legge domanda e chunk insieme.

stesso query set · stessa tabella · una colonna in più



Grazie

prossima lezione: Modulo 3 · Lezione 5 — Advanced Retrieval II: Re-ranking

